

LAB8: Computer Vision

Image Classification using Machine Learning and Convolutional Neural Networks (CNNs)

Objective:

- To implement image classification techniques using traditional machine learning and deep learning (CNNs) models, and to evaluate their performance on standard datasets.

LAB: Computer Vision

Description

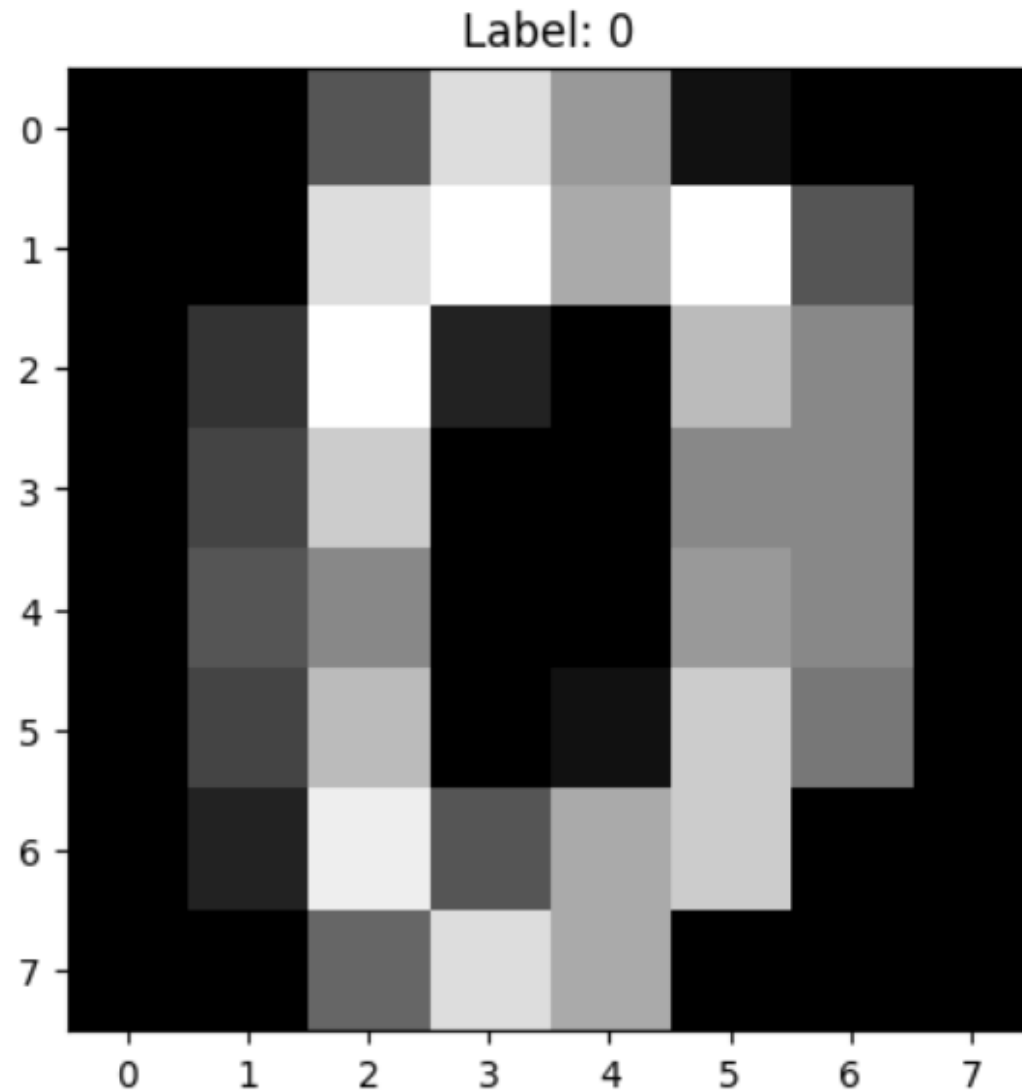
-Carefully observe the key steps for image classification

```
[1]: #Import Libraries  
import numpy as np  
import matplotlib.pyplot as plt  
from sklearn.datasets import load_digits  
from sklearn.model_selection import train_test_split  
from sklearn.metrics import accuracy_score
```

```
[2]: # Load Dataset  
digits = load_digits()  
X = digits.images  
y = digits.target  
  
print("Shape:", X.shape)
```

Shape: (1797, 8, 8)

```
[3]: # Visualize Images  
plt.imshow(X[0], cmap='gray')  
plt.title(f"Label: {y[0]}")  
plt.show()
```



```
[4]: # Preprocessing-Flatten images for ML model
```

```
X = X.reshape((X.shape[0], -1))
```

```
[5]: # Train-Test Split
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)
```

```
[6]: # Train Simple Model (SVM)
```

```
from sklearn.svm import SVC
```

```
model = SVC()
```

```
model.fit(X_train, y_train)
```

```
[6]:
```

▼ SVC ⓘ ?

► Parameters

```
[7]: # Prediction & Accuracy
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
```

Accuracy: 0.9861111111111112

```
[8]: # Deep Learning (Mini CNN using Keras)-Reshape Data
X = digits.images
X = X / 16.0 # normalize

X = X.reshape(-1, 8, 8, 1)
```

```
[9]: # Split Data
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
[10]: # Build CNN Model
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Conv2D, Flatten, Dense

model = Sequential([
    Input(shape=(8, 8, 1)), #  Add this line
    Conv2D(32, (3,3), activation='relu'),
    Flatten(),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])
```

```
[11]: # Compile Model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

```
[12]: # Train Model
model.fit(X_train, y_train, epochs=5)

Epoch 1/5
45/45 ————— 2s 6ms/step - accuracy: 0.6319 - loss: 1.6712
Epoch 2/5
45/45 ————— 0s 6ms/step - accuracy: 0.8900 - loss: 0.5202
Epoch 3/5
45/45 ————— 0s 6ms/step - accuracy: 0.9395 - loss: 0.2508
Epoch 4/5
45/45 ————— 0s 5ms/step - accuracy: 0.9569 - loss: 0.1844
Epoch 5/5
45/45 ————— 0s 7ms/step - accuracy: 0.9582 - loss: 0.1584
```

```
[12]: <keras.src.callbacks.history.History at 0x256a40b4ec0>
```

```
[13]: # Evaluate
loss, acc = model.evaluate(X_test, y_test)
print("CNN Accuracy:", acc)

12/12 ————— 0s 7ms/step - accuracy: 0.9639 - loss: 0.1240
CNN Accuracy: 0.9638888835906982
```

Tasks for Students

1. Display 5 images with labels
2. Train CNN and report accuracy
3. Modify:
 - Increase epochs to 10
 - Add one more Conv layer
4. Plot confusion matrix
5. Write short conclusion

- *At the end create pdf file and submit, don't be late, late submission may affect your sessional marks*